A Unified Framework for Graph Integration in Transformers

22000034 Minha Kwon

1. Introduction

Recent advances in transformer architectures have demonstrated strong performance across various domains, including natural language processing and computer vision. However, their application to graph-structured data remains challenging, primarily due to the lack of inherent structural awareness in standard transformers.

To address this limitation, much research has explored incorporating graph structure into transformer models. These approaches aim to leverage relational information between entities (connectivity, distance, and dependency) to enhance model expressiveness. As a result, graph-based transformers have been successfully applied in diverse domains, including molecular modeling, program analysis, and relational data learning. [1]

Despite these advances, existing studies differ significantly in how graph information is constructed, represented, and integrated into transformer architectures. [2, 3] Many methods focus on specific components, such as positional encoding or attention mechanisms. Other aspects such as integration strategies are often implicitly embedded within model designs. This lack of a unified perspective makes it difficult to systematically compare different approaches and understand their design trade-offs.

Graph integration should be viewed as a sequence of interconnected design decisions rather than a single modeling technique. Based on this perspective, we propose a unified framework that organizes graph-based transformer models into five design dimensions: graph type, graph representation strategy, positional encoding, integration strategy, and model architecture. This framework provides a structured way to analyze how structural information is introduced and utilized within transformer models.

2. Related Work

2.1 Graph Representation Learning

Graph representation learning aims to encode graph-structured data into meaningful vector representations for downstream tasks. [4] Traditional approaches focus on shallow methods such as matrix factorization and random walk-based techniques, including *DeepWalk* and *Node2Vec*, which preserve local proximity through sampling-based strategies. [5]

With the rise of deep learning, **Graph Neural Networks (GNNs)** have become the dominant approach. GNNs update node representations through iterative message passing, where each node aggregates information from its neighbors. Representative models such as *GCN, GraphSAGE*, and *GAT* have demonstrated strong performance in various graph learning tasks.

However, GNN-based methods rely heavily on local neighborhood aggregation, which limits their ability to capture long-range dependencies and complex global structures. This limitation has motivated the development of transformer-based approaches for graph data.

2.2 Graph Transformers

To overcome the limitations of GNNs, transformer architectures have been extended to graph-structured data. Graph transformers leverage the self-attention mechanism to model long-range dependencies and global interactions between nodes. [6] A key challenge in graph transformers is how to incorporate structural information into attention mechanisms.

Existing approaches largely focus on designing positional encoding schemes and modifying attention computation. For example, *Graphormer* [7] integrates structural information such as shortest path distance and centrality into attention bias terms, enabling effective global modeling without restricting attention. Other works use spectral encoding methods based on Laplacian eigenvectors or structural encodings to improve expressiveness. [2, 8, 9] While these studies provide strong insights into encoding techniques and architectural design, they typically focus on individual components such as positional encoding or attention modification.

Beyond model architecture, recent studies highlight the importance of how graph data is constructed and integrated. Research on program analysis demonstrates that the choice of graph structures such as AST, CFG, and DFG, has a significant impact on model performance. [3, 10, 11]

Hybrid graph representations that combine multiple views have been shown to improve learning effectiveness by capturing complementary structural information. [12] However, unlike positional encoding or attention mechanisms, these aspects are not systematically categorized in prior work. Graph construction, representation Strategy, and integration strategies are often embedded within specific model designs, making it difficult to compare different approaches in a unified manner.

Existing work on graph-based transformers can be broadly divided into three directions: graph representation learning, graph transformer architecture, and graph construction techniques. [6] While prior studies provide detailed analysis of specific components such as encoding and attention mechanisms, they lack a unified framework that systematically organizes the full design space of graph integration. [3]

3. Graph Integration Framework for Transformers

Graph-based modeling has been widely adopted to enhance transformer architectures by incorporating structural information. However, existing approaches differ significantly in how graph data is constructed and integrated, often presented in isolated forms. This makes it difficult to systematically compare methods or understand their design trade-offs.

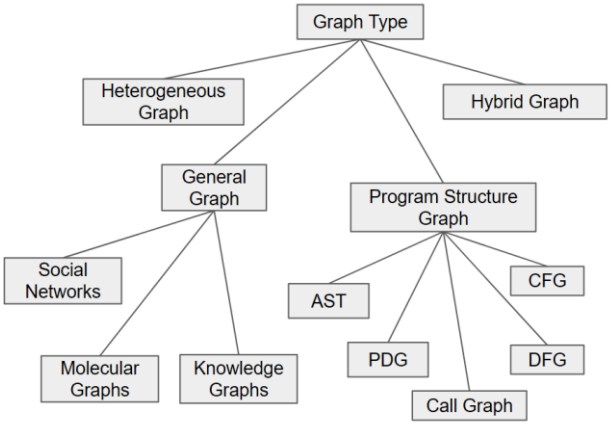
A key challenge is that graph integration is not a single design decision, but a sequence of interconnected choices. Therefore, we organize graph-based transformer models into five design dimensions: graph type, graph representation strategy, positional encoding, integration strategy, and model architecture. These dimensions correspond to different stages in the pipeline of incorporating graph structure into transformer models, from defining the graph itself to integrating structural information into attention mechanisms. Therefore, we provide a unified framework for analyzing graph-transformer models.

3.1 Graph type

Graph type defines the form of relational structure used as input to the model. It determines

what kinds of relationships are available for learning and therefore acts as the foundation for all subsequent design choices. Even when the same data is used, different graph constructions can lead to significantly different performance, because each graph type captures a different aspect of the underlying structure.

In this work, graph types are organized into four main categories: general graphs, program structure graphs, heterogeneous graphs, and hybrid graphs.



3.1.1 General Graph

General graphs represent relationships between entities in a domain-independent manner. Nodes correspond to entities, and edges represent their interactions. This is the most common graph type in graph learning and is widely used across various applications.

Typical examples include social networks, molecular graphs, and knowledge graphs. In social networks, nodes represent users and edges represent relationships such as friendships or interactions. [4, 5] In molecular graphs, nodes correspond to atoms and edges to chemical bonds, allowing models to learn chemical properties. Knowledge graphs represent entities and their semantic relationships, such as "is-a" or "part-of."

General graphs are flexible and widely applicable, but they often rely on the assumption that all nodes and edges are homogeneous, which may limit their ability to capture more complex structures.

3.1.2 Program Structure Graph

Program structure graphs are designed to represent the internal structure of source code. Unlike

general graphs, which describe relationships between entities, these graphs explicitly capture syntactic and semantic structures within programs. [10, 11, 13]

Abstract Syntax Tree (AST) represents the syntactic structure of code based on grammar rules. **Control Flow Graph (CFG)** models execution flow, capturing how control moves through the program. **Data Flow Graph (DFG)** represents how values are produced, used, and propagated. **Program Dependence Graph (PDG)** combines control and data dependencies into a unified representation. **Call Graph** captures relationships between functions or methods through invocation links. Each graph type highlights a different aspect of program behavior. In practice, these graphs are often used together to provide complementary views of the code. [12]

3.1.3 Heterogeneous Graph

Heterogeneous graphs extend general graphs by allowing multiple types of nodes and edges. [3, 6] This enables the modeling of complex systems where different entities and relationships coexist. For example, a recommendation system may include users, items, and categories as different node types, with edges representing interactions such as "purchased" or "belongs to." Similarly, knowledge graphs often use heterogeneous structures to represent rich semantic relationships. Heterogeneous graphs provide higher expressive power, but they also introduce additional complexity in model design, as different types of nodes and edges must be handled separately.

3.1.4 Hybrid Graph

Hybrid graphs combine multiple graph types into a single representation. Instead of relying on a single structural view, hybrid graphs integrate complementary information from different graph sources. A common example is code analysis, where AST, CFG, and DFG are combined to capture syntax, control flow, and data flow simultaneously. [10, 11] This multi-view representation allows the model to access richer structural information than any single graph alone. [12] Hybrid graphs generally lead to improved performance because they capture diverse relationships. However, they also increase model complexity and computational cost, requiring careful integration of multiple graph structures. [14]

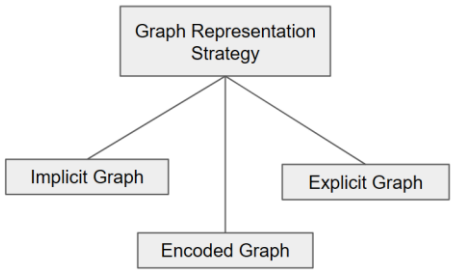
Graph type determines what relationships are visible to the model and therefore directly influences all later stages, including representation, positional encoding, integration strategy, and model architecture. While general graphs provide broad applicability, program structure graphs offer domain-specific structural detail, and heterogeneous and hybrid graphs enable richer and

more expressive representations.

3.2 Graph Representation Strategy

Graph representation strategy defines how graph structure is utilized within a model. It goes beyond simply using a graph as input and determines whether the structure is treated as a computational process, an explicit feature, or a set of derived numerical signals. As a result, it plays a critical role in shaping how structural information is propagated, interpreted, and integrated throughout the model.

In this work, graph representation strategies are categorized into three types: implicit, explicit, and encoded representations. These categories are distinguished by the degree to which graph structure is directly exposed to the model and how it influences computation.



3.2.1 Implicit Representation

Implicit representation utilizes graph structure indirectly through the computational process. [15] In this setting, the graph is not explicitly provided as a feature but instead defines the pathway through which information flows.

A representative example is the **graph neural networks (GNNs)**, where node representations are updated through iterative message passing. [13] Each node aggregates information from its neighbors, and the graph structure determines how information propagates across the network. In this case, the graph acts as a constraint on computation rather than an input feature.

The main advantage of implicit representation is that it naturally incorporates structure without requiring additional feature engineering. However, its effectiveness is limited by the depth of propagation, which can make it difficult to capture long-range dependencies. [15]

3.2.2 Explicit Representation

Explicit representation treats graph structure as an independent source of information by converting it into features that are directly fed into the model. In this approach, the graph topology is transformed into node-level or graph-level representations, which are then combined with other input features.

For example, methods such as network embedding transform graph structure into vector representations that can be used alongside node features. Models such as *Graph Explicit Neural Networks (GENN)* explicitly encode topology in this way, allowing the model to leverage structural information even when node features are limited. [16]

This approach provides clear and direct access to structural information and can improve robustness in feature-scarce settings. However, its performance depends heavily on how the structure is encoded, and the preprocessing step may introduce additional complexity.

3.2.3 Encoded Representation

Encoded representation converts graph structure into numerical signals that are integrated into the model without preserving the original graph form. Instead of using the graph directly, structural information is summarized through quantities such as distances, centrality measures, or edge attributes.

A typical example is *Graphormer* [7], where shortest path distance, node centrality, and edge features are incorporated as bias terms in the attention mechanism. In this case, the graph is not used as a structure during computation; instead, its properties are encoded into numerical values that influence attention weights.

This approach allows graph information to be integrated into transformer models without modifying the underlying architecture. It is particularly effective for capturing global relationships. However, since the structure is compressed into numerical features, some structural information may be lost.

Graph representation strategies define how graph structure is introduced into the model, ranging from implicit computational use to explicit feature representation and encoded numerical integration. These choices affect how structural information is utilized and directly influence the design of positional encoding, integration strategies, and model architecture. In summary, the key distinction lies in whether the graph is used as a computational pathway, an explicit input, or a source of numerical signals derived.

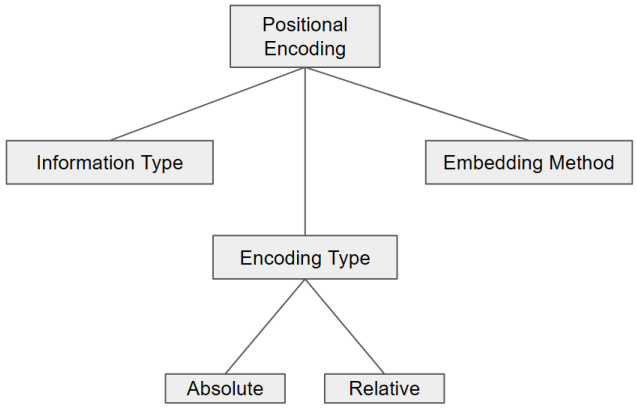
3.3 Positional Encoding

Positional encoding is a key component in transformer-based models for graph data, designed to incorporate structural information into node representations. Unlike sequences, graphs do not possess an inherent ordering of elements, making it necessary to explicitly encode relationships such as connectivity, distance, or global structure. [6] Through this process, positional encoding enables the transformer to operate on graph-structured inputs by providing the missing notion of relational context.

However, positional encoding is not universally required in all graph transformer models. In some approaches, structural information is integrated directly into the attention mechanism without constructing explicit positional embeddings. For example, models such as *Graphormer* [7] inject graph structure by adding distance and centrality information as biases in the attention computation, rather than encoding them as standalone positional vectors. [3] In these cases, positional information is still utilized, but it is embedded implicitly within the attention scores instead of being represented as explicit node-level features. [15]

Therefore, positional encoding should be understood not as a fixed module, but as a broader design space for incorporating structural information into transformer models. This includes both explicit embedding-based encodings and implicit mechanisms integrated into attention.

In this work, positional encoding is characterized along three complementary dimensions: **encoding type, information type, and embedding method**. Together, these dimensions describe what structural information is used, how relationships are represented, and how this information is transformed into a format compatible with transformer architectures.



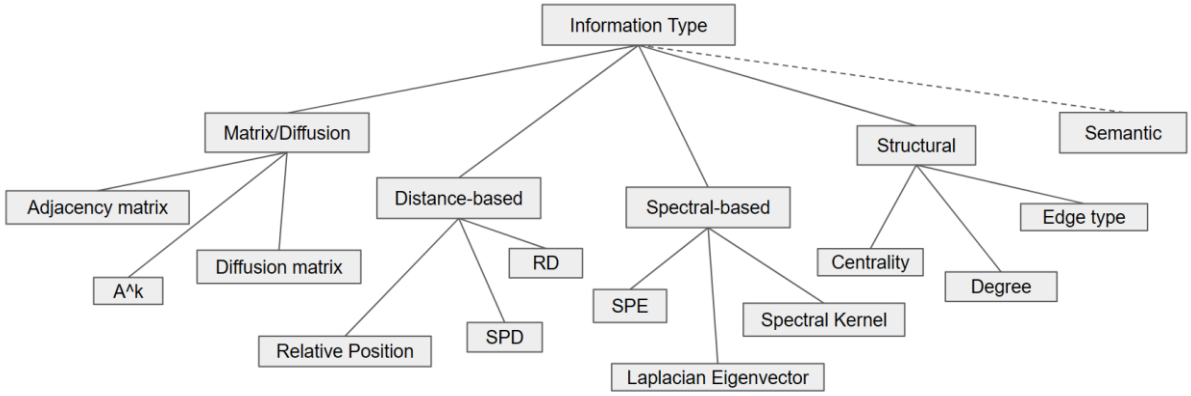
3.3.1 Encoding Type

Encoding type defines the granularity at which positional relationships are represented.

Absolute encoding assigns a distinct vector to each node independently, capturing node-specific properties. In contrast, **relative encoding** describes relationships between pairs of nodes, typically using distance or connectivity information. In graph domains, relative encoding is generally more effective because structural relationships between nodes carry more information than absolute node positions. Many graph transformer models therefore rely primarily on relative positional encoding schemes. [17]

3.3.2 Information Type

Information type determines which structural properties of the graph are encoded. It is the most critical factor influencing the expressive power of positional encoding.



(1) Distance-based [17] approaches encode pairwise relationships using explicit distance metrics. **Shortest Path Distance (SPD)** measures the minimum number of edges between two nodes and is widely used due to its simplicity and effectiveness. **Resistance Distance (RD)** models the graph as an electrical network, capturing global connectivity patterns beyond shortest paths. **Relative position encodings** often discretize distances into buckets to provide coarse relational information. [3]

(2) **Structural** approaches focus on node roles and connectivity patterns rather than pairwise distances. **Degree** encodes the number of connections a node has, while **centrality** measures quantify the importance of nodes within the graph. **Edge types** distinguish different types of relationships between nodes. [13] These features capture functional roles rather than geometric relationships. [3]

(3) **Spectral-based** methods leverage the eigen structure of the graph Laplacian to encode global topology. [9] **Laplacian eigenvectors** provide continuous positional embeddings

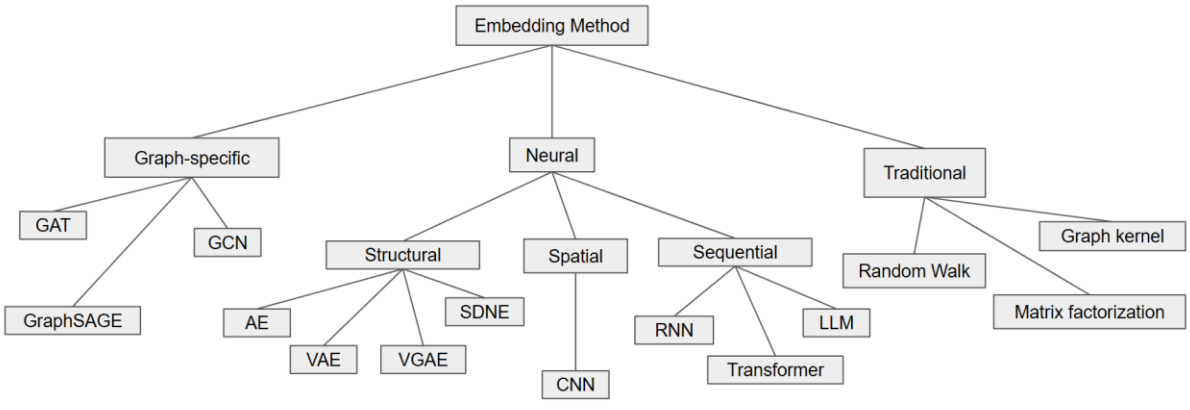
that reflect the overall structure of the graph. [9] **Spectral kernels** extend this idea by incorporating multi-scale structural information through eigenvalues. **Structural Positional Encoding (SPE)** builds on spectral representations to generate node embeddings that combine both global and local structure. [3, 18]

(4) **Matrix and diffusion-based** methods treat matrix representations of the graph as the primary information source. The **adjacency matrix** captures direct connections between nodes, while higher-order powers such as A^krepresent multi-hop relationships. **Diffusion matrices** model information propagation processes such as random walks. These approaches focus on how information flows through the graph rather than static structural properties. [3]

(5) **Semantic information** extends beyond structural encoding by incorporating contextual or domain-specific knowledge. [11] Examples include language model embeddings or retrieved contextual features. Although semantic features do not strictly represent positional structure, they are increasingly used as an extension to enrich relational understanding in modern graph transformer systems.

3.3.3 Embedding Method

Embedding methods describe how the selected structural information is converted into vector representations that can be consumed by neural models. Unlike information type, which defines what is encoded, embedding methods define how encoding is performed. [5]



Embedding Method [5]

(1) **Traditional methods** rely on mathematical and statistical techniques to generate embeddings. [5] **Matrix factorization** decomposes adjacency or similarity matrices into

low-dimensional representations that preserve structural relationships. **Random walk-based** methods such as *DeepWalk* and *Node2Vec* generate sequences of nodes and apply language modeling techniques to learn embeddings. **Graph kernels** compute similarity measures between graph structures, which can be used to derive embeddings. These methods are generally efficient but limited in capturing highly complex non-linear patterns. [5]

(2) **Neural methods** apply general deep learning architectures to graph data. [5] **Autoencoder-based models (structural)** compress and reconstruct graph structures to learn compact representations. **Autoencoders (AE)** learn deterministic encodings, while **variational autoencoders (VAE)** introduce probabilistic representations. Models such as **SDNE** and **VGAE** are specifically designed to preserve graph structure during reconstruction. **Sequence-based** methods transform graph data into sequences, typically through random walks, and process them using **recurrent neural networks (RNNs)** or **transformers**. **Spatial** approaches such as **convolutional neural networks (CNNs)** treat graph neighborhoods as spatial patterns and learn local structural features. [5]

(3) **Graph-specific methods** directly incorporate graph topology into the model architecture.[5] **Graph Convolutional Networks (GCN)** aggregate information from neighboring nodes to update node representations. **Graph Attention Networks (GAT)** introduce attention mechanisms to weigh neighbor contributions dynamically. **GraphSAGE** employs sampling strategies to scale neighborhood aggregation to large graphs. These methods explicitly model graph structure and are widely used in modern graph learning tasks.

Positional encoding in graph transformers is not a single technique but a composition of design choices. It determines how structural information is extracted, represented, and integrated into the model. The overall effectiveness of a graph transformer depends on how well these components are chosen and combined.

In this sense, positional encoding can be viewed as the central mechanism that translates graph structure into a form compatible with attention-based models, bridging the gap between discrete graph topology and continuous vector representation.

3.4 Integration Strategy

Integration strategy refers to how graph structure is incorporated into transformer models.

Beyond selecting which structural information to use, it is equally important to determine where and how this information is injected into the model. The same structural signals can lead to very different behaviors depending on their integration point.

In this work, integration strategies are organized based on how deeply graph structure is incorporated into the transformer, ranging from external preprocessing to internal relational modeling.

3.4.1 Graph Tokenization

Graph tokenization transforms graph structures into tokens that can be directly processed by transformers. In this approach, nodes, edges, or subgraphs are treated as tokens, and the graph is converted into a sequence or a set of tokens. [2, 3] This method allows the use of standard transformer architecture without major modification. However, structural information may be partially lost during tokenization, especially when complex graph topology is simplified into linear or set representations. Typical examples include node-token transformers and models that linearize graph structures.

3.4.2 Input-level Integration

Input-level integration incorporates graph structure into node embeddings before attention is applied. Structural features such as positional encodings or graph embeddings are added to node features at the input stage. [9] This approach is simple to implement and does not require modifying the transformer architecture. However, because structural information is not explicitly used during attention computation, the model may have limited ability to capture complex relationships. This strategy is often used when positional encoding is treated as an input feature.

3.4.3 Attention-level Integration

Attention-level integration directly incorporates structural information into the attention score computation. Instead of modifying the input, this approach adjusts how attention weights are calculated between nodes. A common method is to add structural features, such as distance or connectivity, as bias terms in the attention scores. In this way, all nodes can still attend globally, but their importance is adjusted according to graph structure. A representative example is Graphormer [7], which uses shortest path distance, node centrality, and edge information as attention biases. This approach provides a flexible way to incorporate structure while preserving

global attention.

3.4.4 QKV-level Integration (Relational Integration) [19]

QKV-level integration incorporates graph structure into the query, key, and value representations used in attention. [8] In this approach, structural information directly affects how attention is computed at the representation level. Edge information is often used to model relationships between node pairs, allowing the model to capture more expressive interactions than distance-based approaches. This method is commonly referred to as relational attention, as it explicitly models pairwise relations between nodes. [20] While it offers stronger expressive power, it also increases computational complexity.

3.4.5 Edge-level Integration

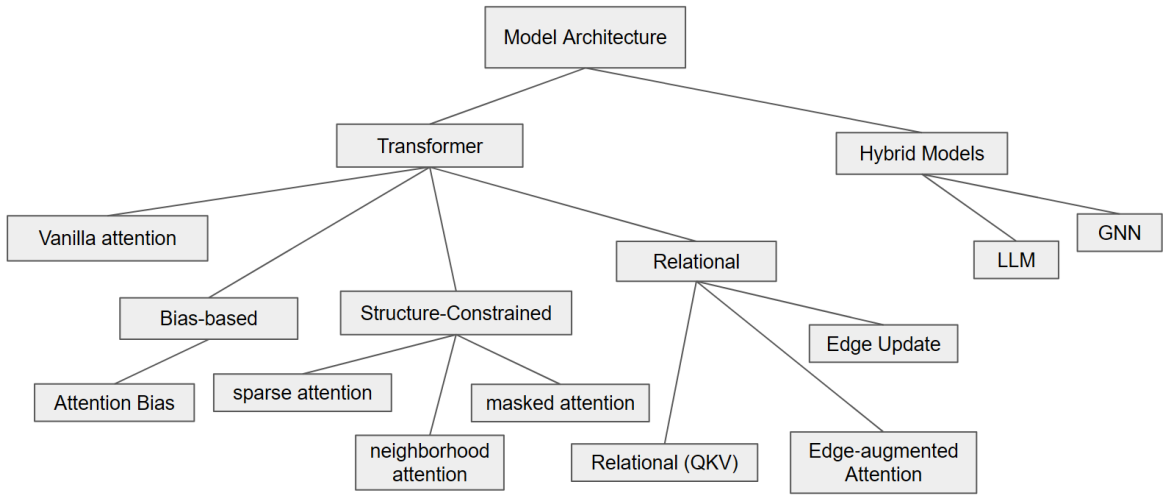
Edge-level integration extends the model to jointly learn node and edge representations. [18] Instead of treating edges as auxiliary signals, this approach models them as independent entities that evolve during training. The model updates both node and edge embeddings, enabling it to capture more complex relational structures and higher-order dependencies. This approach is typically used in relational transformer models and provides the highest expressiveness but also introduces the greatest computational cost.

Strategy	Integration Level	Representative Examples
Graph Tokenization	Pre-processing	Node-token transformer, linearized graph models
Input-level Integration	Input Embedding	PE-based models, feature augmentation
Attention-level Integration	Attention Score	Graphormer [7]
QKV-level Integration	Attention Representation	Relational Attention[20], edge-aware QKV
Edge-level Integration	Full Model (Node + Edge)	Relational Transformer, edge update models [3]

Integration strategies define where and how graph structure interacts with transformer computation. They range from external representations, such as tokenization, to deeply integrated relational modeling at the edge level. Overall, they can be viewed as a progression from indirect and simple integration to more explicit and expressive modeling of graph structure.

3.5 Model Architecture

Model architecture defines how graph structure is incorporated into transformer-based models. Broadly, existing approaches can be divided into two categories: transformer-based models and hybrid models. These two categories differ in whether graph structure is handled directly within attention mechanisms or through a combination of multiple models.



3.5.1 Transformer-based Models

Transformer-based models retain the original self-attention framework and modify it to incorporate graph structure. In this setting, the key design question is how structural information influences the attention computation. Based on this perspective, attention mechanisms can be categorized into several types. [2, 6]

(1) **Vanilla Attention** uses the standard self-attention mechanism without explicitly incorporating graph structure. All nodes attend to each other in a fully connected manner. This approach does not exploit structural information, so its expressive power is limited for graph data. However, it serves as a useful baseline and is sometimes used when structural signals are provided through other components such as features or implicit encoding. Typical examples include standard transformer models applied to graph inputs without structural bias.

(2) **Structure-constrained attention** incorporates graph structure by restricting which node pairs can attend to each other. Instead of allowing full connectivity, attention is computed

only over selected edges or neighborhoods.

This category includes several closely related variants: **Masked attention** uses an adjacency mask to allow attention only between connected nodes. [1] **Sparse attention** reduces computational cost by selecting a subset of node pairs. [6] **Neighborhood attention** limits attention to local k-hop neighbors.

Although the implementation differs, all these methods share the same principle: limiting the attention range based on graph structure. This approach is efficient and captures local patterns effectively, but it may struggle to model long-range dependencies.

(3) **Bias-based attention** incorporates graph structure by adding structural information directly to the attention scores. Instead of restricting connections, it adjusts the importance of each pair of nodes. Each node pair can still attend globally, but its attention weight is modified by factors such as distance or centrality. A representative example is *Graphormer* [7], where structural features such as shortest path distance, node centrality, and edge types are encoded as attention bias terms. This approach preserves global attention while incorporating graph structure in a soft and flexible way.

(4) **Relational attention** explicitly models edge information within the attention mechanism. Unlike bias-based methods, which treat structure as a scalar adjustment, relational methods treat edges as first-class elements in the computation. [20]

Edge-augmented attention incorporates edge features as additional inputs during attention computation. [18] **Relational attention (Q, K, V)** modifies the query, key, and value representations using edge information, enabling relation-aware interactions. [8, 20] **Edge update** approaches maintain and update both node and edge representations during learning.

These methods provide the most expressive modeling of graph structure, as they directly encode relationships between nodes. However, they also increase model complexity and computational cost. Typical examples include relational transformers and edge-aware attention models.

3.5.2 Hybrid Models

Hybrid models combine transformers with other architectures to better capture graph structure. Instead of relying solely on attention, these models leverage complementary strengths from different learning paradigms.

(1) **Transformer with GNN**

Graph neural networks (GNNs) are naturally suited for modeling local neighborhood structures. When combined with transformers, GNNs capture local patterns while transformers model global relationships. [8] Several integration strategies are possible: For sequential integration, GNN layers are applied before the transformer. For parallel, GNN and transformer operate simultaneously and their outputs are combined. [3] Interleaved integration is also possible. This combination improves both local and global representation but increases model complexity.

(2) **Transformer with LLM**

Recent approaches integrate large language models (LLMs) with graph transformers to incorporate semantic information. In this setting, node representations may include textual or contextual features derived from language models. This design allows the model to capture both structural and semantic relationships. It is especially useful for applications where nodes have rich textual descriptions or external knowledge sources. However, these models are typically more resource-intensive and require careful alignment between structural and semantic information.

Model architecture in graph transformers is primarily determined by how attention mechanisms are modified to incorporate graph structure. Transformer-based approaches adjust attention directly, while hybrid approaches combine transformers with other models to enhance structural modeling.

In this context, attention mechanisms can be viewed as the core component that defines how graph information is integrated. Different choices—restricting connections, adjusting attention weights, or explicitly modeling relations—lead to different architectural families with distinct strengths and trade-offs.

4. Case Analysis and Discussion

4.1 Representative Methods Mapping

To validate the proposed framework, we analyze representative graph-transformer models by mapping them onto the five design dimensions: graph type, representation strategy, positional

encoding, integration strategy, and model architecture. This mapping illustrates how different design choices are combined in existing methods. The following table summarizes several representative models.

Model/Type	Graph Type	Representation	Positional Encoding	Integration	Architecture
Graphormer [7]	Molecular Graph	Encoded	None	Attention-level	Attention Bias
Allamanis (GGNN) [13]	AST, CFG, DFG	Implicit + Explicit	Structural (edge type)	Input-level	Hybrid (GNN)
RelGT [19]	Heterogeneous Graph	Explicit	Relative distance + type encoding	QKV-level + Tokenization	Relational
GENN [16]	General Graph	Explicit	None	Input-level	Hybrid

4.2 Observations and Insights

From the above mapping, several patterns can be observed. First, many models follow two dominant design directions. One direction focuses on encoded representations combined with attention-level integration, as seen in *Graphormer* [7]. This approach allows efficient incorporation of global structural information without modifying the core architecture. The other direction emphasizes explicit modeling of relations through QKV-level or edge-level integration, which provides richer expressiveness but increases complexity. [19]

Second, there is a clear trade-off between simplicity and expressiveness. Input-level and bias-based methods are computationally efficient and easy to implement, but they rely on indirect structural modeling. In contrast, relational and edge-aware methods directly model interactions between nodes, leading to better representation power at the cost of higher computational overhead. [14]

Third, the choice of graph types strongly influences downstream design decisions. Models working on program structure graphs often adopt hybrid or explicit representations to preserve detailed structural relationships, while general graph models more frequently use encoded representations for scalability.

5. Conclusion

Graph-based transformers have shown strong potential by incorporating structural information into attention-based models. However, their design space is complex and often fragmented across different studies.

In this work, we proposed a unified framework that organizes graph integration into five key dimensions: graph type, representation strategy, positional encoding, integration strategy, and model architecture. This framework provides a structured perspective for understanding and comparing existing approaches. Different models can be interpreted as combinations of these design choices. And we demonstrated important trade-offs between simplicity, efficiency, and expressiveness.

This framework can contribute when analyzing and selecting appropriate design for graph-transformer models.

6. Reference

[1] Buterez, D., Janet, J. P., Oglic, D., & Liò, P. (2025). An end-to-end attention-based approach for learning on graphs. *Nature Communications*, *16*(1), 5244.

[2] Müller, L., Galkin, M., Morris, C., & Rampášek, L. (2023). Attending to graph transformers. *arXiv preprint arXiv:2302.04181*.

[3] Yuan, C., Zhao, K., Kuruoglu, E. E., Wang, L., Xu, T., Huang, W., ... & Rong, Y. (2025). A survey of graph transformers: Architectures, theories and applications. arXiv preprint arXiv:2502.16533.

[4] Ju, W., Fang, Z., Gu, Y., Liu, Z., Long, Q., Qiao, Z., ... & Zhang, M. (2024). A comprehensive survey on deep graph representation learning. *Neural Networks*, *173*, 106207.

[5] Khoshraftar, S., & An, A. (2024). A survey on graph representation learning methods. *ACM Transactions on Intelligent Systems and Technology*, *15*(1), 1-55.

[6] Shehzad, A., Xia, F., Abid, S., Peng, C., Yu, S., Zhang, D., & Verspoor, K. (2026). Graph transformers: A survey. *I16 Transactions on Neural Networks and Learning Systems*.

[7] Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., ... & Liu, T. Y. (2021). Do transformers really perform badly for graph representation?. *Advances in neural information processing systems*, *34*, 28877-28888.

[8] Chen, D., O'Bray, L., & Borgwardt, K. (2022, June). Structure-aware transformer for graph representation learning. In *International conference on machine learning* (pp. 3469-3489). PMLR.

[9] Kreuzer, D., Beaini, D., Hamilton, W., Létourneau, V., & Tossou, P. (2021). Rethinking graph transformers with spectral attention. *Advances in neural information processing systems*, *34*, 21618-21629.

[10] Liu, J., Zeng, J., Wang, X., & Liang, Z. (2023, May). Learning graph-based code representations for source-level functional similarity detection. In *2023 I16/ACM 45th International Conference on Software Engineering (ICSE)* (pp. 345-357). I16.

[11] He, H., Lin, X., Weng, Z., Zhao, R., Gan, S., Chen, L., ... & Xue, Z. (2024). Code is not natural language: Unlock the power of {Semantics-Oriented} graph representation for binary code similarity detection. In *33rd USENIX Security Symposium (USENIX Security 24)* (pp. 1759-1776).

[12] Zhang, Y., Zhu, J., Yang, Y., Wen, M., & Jin, H. (2023, August). Comparing the performance of different code representations for learning-based vulnerability detection. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware* (pp. 174-184).

[13] Allamanis, M., Brockschmidt, M., & Khademi, M. (2017). Learning to represent programs with graphs. *arXiv preprint arXiv:1711.00740*.

[14] Zhang, Z., & Saber, T. (2025, September). Ast-enhanced or ast-overloaded? the surprising impact of hybrid graph representations on code clone detection. In *2025 I16 International Conference on Software Maintenance and Evolution (ICSME)* (pp. 271-281). I16.

[15] Gu, F., Chang, H., Zhu, W., Sojoudi, S., & El Ghaoui, L. (2020). Implicit graph neural networks. *Advances in neural information processing systems*, *33*, 11984-11995.

[16] Wang, Y., Hooi, B., Liu, Y., & Shah, N. (2023, February). Graph explicit neural networks: Explicitly encoding graphs for efficient and accurate inference. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining* (pp. 348-356).

[17] Black, M., Wan, Z., Mishne, G., Nayyeri, A., & Wang, Y. (2024). Comparing graph transformers via positional encodings. *arXiv preprint arXiv:2402.14202*.

[18] Hussain, M. S., Zaki, M. J., & Subramanian, D. (2021). Edge-augmented graph transformers: Global self-attention is enough for graphs. *arXiv preprint arXiv:2108.03348*, *3*.

[19] Dwivedi, V. P., Jaladi, S., Shen, Y., López, F., Kanatsoulis, C. I., Puri, R., ... & Leskovec, J. (2025). Relational graph transformer. *arXiv preprint arXiv:2505.10960*.

[20] Diao, C., & Loynd, R. (2022). Relational attention: Generalizing transformers for graph-structured tasks. *arXiv preprint arXiv:2210.05062*.